COFA 실전 운영안 · 바로 복붙하는 명령 모음

내 작업 방식: 핵심 흐름을 먼저 연결하고, 독립 작업은 병렬화하고, 검증 결과로 통합한다. 개발하면서 선택 이유와 한계를 이해한다.

사용 순서: 처음에는 아래 '1. 시작'의 코드 블록만 복사한다. 이후에는 해당 상황의 블록 하나만 사용한다. 문서 전체를 한 번에 입력할 필요는 없다. [남은 시간]을 실제 값으로 바꾸고, 모르면 타이머 확인 후 알려준다.

파일 업로드 대신 공개 저장소를 과제 폴더 아래 .cofa-playbook으로 clone해 사용한다. 추가 스킬 설치는 필요 없다. 제공 프로젝트·현재 응시 지침을 먼저 따른다. 실제 서술형 답변 단계가 아니라 구현 및 허용된 사전 연습을 위한 운영안이다.

1. 시작 · 가장 먼저 붙여넣기

이번 과제의 목표는 필수 요구를 충족하는 결과물을 완성하고,
내가 실제 구조·선택 이유·한계를 직접 설명할 수 있게 하는 것이다.
현재 남은 시간은 [남은 시간]이다. 너는 메인 구현자와 통합 책임자다.

먼저 공식 README·AGENTS·제공 파일·기존 변경·실행/제출 방법을 확인해라.
기존 스택을 우선 활용하고, 파일 구조를 파악한 뒤
입력→검증→핵심 처리→저장→최종 결과의 연결을 집중 분석해라.
확인된 사실·추가 가정·결과를 바꾸는 미확인을 구분해라.
업무상 중요한 미확인만 질문하고 관련 없는 작업은 계속해라.
일반적인 기술 선택은 이유를 기록하고 진행해라.

기존 계획이 있으면 갱신하고, 없으면 PLAN.md를 만들어라.
필수 요구 ID·수락 조건·구현 순서·담당 파일·의존성·검사 방법을 적어라.
작은 입력부터 최종 결과까지 연결되는 첫 흐름을 실제로 실행한 뒤 확장해라.
첫 흐름은 중간 이정표이며 나머지 필수 요구도 끝까지 구현해라.

메인은 핵심 흐름·공유 계약·공통 설정·통합과 공통 기록을 직접 맡아라.
독립 작업이 생기면 구현 서브에이전트 1~2명을 우선 사용해라.
각자 목표·편집 범위·입출력 계약·완료 조건·검사·반환 형식을 줘라.
워크트리는 실제 지원되고 격리가 유리할 때 사용해라.
공유 파일·스키마·의존성은 한 담당자가 조정하고,
내장 서브에이전트가 없으면 같은 작업 순서를 직접 수행해라.

첫 핵심 흐름과 최종 통합본은 독립 검토자에게 확인시켜라.
검토자는 요구 누락·계산 오류·예외·데이터 보존·실행 실패를 우선하고,
파일 위치·재현 조건·기대/실제 결과를 제시해야 한다.
타당한 지적은 원 구현자가 수정하고 관련 검사를 다시 실행해라.
최종 검사는 실제 제출 루트에서 수행해라.

작업자는 변경·선택 이유·코드 위치·실제 검사·한계를 반환해라.
메인은 기존 기록 또는 PROJECT_DEFENSE.md에 중요한 결정을 짧게 남겨라.
완료·막힘·공유 계약 변경은 즉시 알리고,
대략 5분마다 완료한 요구·실행 근거·위험·다음 행동을 요약해라.
보고를 기다리는 동안에도 독립 작업을 계속해라.

중요 흐름이 완성되면 작은 입력 한 건으로 처리 과정과 선택 이유를 설명하고,
내가 다른 입력의 결과를 설명할 질문 하나를 제시해라.
내 답변 대기 때문에 독립 개발을 멈추지 마라.
마지막에는 배포·모듈·데이터 흐름·알고리즘·설정 변경·실패 대응·가정을
실제 코드로 정리하고 내가 복습할 시간을 남겨라.
먼저 현재 상태, 첫 실행 방법, 첫 구현 흐름을 짧게 보고하고 진행해라.

2. 전체 파이프라인 · 내가 개입하는 지점

시간은 공식 제한이 아닌 운영 예시다. 남은 시간의 마지막 15~20%는 통합 검증·실행 확인·설명 복습에 남긴다. 배포나 미리보기 위험은 처음부터 작은 실행으로 확인한다.

단계	에이전트가 해야 할 일	내가 확인할 것	다음 단계의 증거
파악	요구·데이터·실행·제출 위치 확인	업무 규칙과 추가 가정	대표 입력·기대 결과·첫 실행
계획	요구와 검사 연결, 의존성 정리	필수/선택 범위	짧은 PLAN
첫 구현	입력부터 최종 결과까지 연결	실제 사용자 행동이 되는가	실행 결과와 검사
확장	독립 작업 분할, 필수 기능 완성	중요한 변경·막힘	작업별 코드와 검사 근거
통합 검토	결함 재현·최소 수정·재검사	남은 필수 요구와 큰 위험	제출 루트의 연결 검사
설명 준비	실제 구현·한계 정리, 리허설	내가 내 말로 설명 가능한가	직접 답변과 보충 학습

중요한 업무 답을 전달할 때

```
[질문/요구 ID]에 대한 실제 답변은 다음과 같아: [답변 원문].
관련 요구·수락 조건·가정을 갱신하고 영향받는 코드와 검사를 찾아라.
담당 작업자에게 변경을 전달하고 적용 결과까지 확인해라.
이 답변을 넘어선 내용은 고객이 확정한 사실로 만들지 마라.
독립 작업은 계속 진행해라.
```

첫 흐름이 동작했을 때

```
지금 구현된 흐름을 작은 입력 한 건으로 설명해라.
입력값이 어떤 함수와 규칙을 거쳐 어떤 최종 결과가 되는지 보여줘.
요구 충족·검증 완료·미검증을 구분하고 다음 필수 요구를 이어서 구현해라.
```

보고는 “80% 완료” 대신 “R02 저장 후 재조회 확인, R03 오류 처리 진행, 재시작 보존 미검증”처럼 받는다. 5분은 묶음 보고 기준이며, 실패나 계약 변경은 즉시 전달한다.

3. 병렬 구현 · 나눌 준비가 됐을 때

기본 구성은 메인 1명과 독립 구현자 1~2명이다. 검토자는 완료된 묶음에 호출한다. 수는 고정 목표가 아니며, 작은 과제나 강하게 연결된 작업은 메인이 직접 처리한다.

병렬화를 시작할 명령

```
PLAN의 의존성을 보고 지금 독립적으로 실행할 수 있는 작업을 골라라.
공통 입력·응답·오류·상태 계약을 먼저 확정하고 작업자에게 전달해라.
메인은 핵심 흐름과 통합을 맡고, 독립 작업 최대 2개부터 배정해라.
작업별 편집 범위를 정하고 공통 설정·락파일·스키마 담당을 하나로 정해라.
서브에이전트는 추가 서브에이전트를 만들지 마라.

작업 계약에 다음을 포함해라:
요구 ID / 목표 / 읽을 파일 / 편집 허용 파일 / 입출력 계약 /
수락 조건 / 검사 방법 / 변경 금지 범위 / 반환할 근거.

워크트리를 사용하면 정확한 경로·브랜치·기준 커밋을 전달해라.
분리한 테스트 DB·서버 포트·환경 설정을 확인하고,
완료된 변경은 메인이 검토해 실제 제출 루트로 통합해라.
지원이 없거나 통합 비용이 커지면 단일 작업공간의 파일 소유 방식으로 진행해라.
```

분할 판단 예시

분할하려는 작업	판단
확정된 API를 쓰는 결과 화면 / 별도 입력 검증 함수	병렬 가능
계산 모듈 구현 / 읽기 전용 반례 조사	병렬 가능
API 스키마 설계 / 스키마를 추측하는 프론트 구현	계약부터 확정
같은 데이터 모델을 수정하는 두 기능	단독 담당·순차 통합
모두가 공통 패키지·락파일·DB 마이그레이션 수정	메인 또는 지정 담당자만

작업자 반환 형식

```
작업 ID와 요구 ID:
변경 파일·핵심 함수:
실제 선택과 이유:
실행한 검사·결과:
미구현·미검증·한계:
통합할 변경/커밋과 필요한 연결:
```

상시 기록 전담 대신 각 작업자가 근거를 반환하고 메인이 공통 문서를 갱신한다. 마지막에는 다른 에이전트에게 문서와 코드의 일치 여부를 검사시킬 수 있다.

4. 검토와 수정 · 완성된 변경 묶음을 넘긴다

첫 핵심 흐름이 실제 동작했을 때 한 번, 필수 기능을 통합한 뒤 한 번 검토한다. 작업 중인 모든 파일을 지속 감시시키기보다 검토 범위와 시점을 특정한다.

검토 명령

```
독립 검토 에이전트에게 현재 완료된 변경 묶음을 읽기 전용으로 맡겨라.
검토 기준은 공식 요구·수락 조건·대표 입력과 실제 코드다.
대상 커밋/변경 파일을 특정하고 관련 미추적 코드도 포함해라.

요구 누락, 잘못된 계산, 예외 처리, 설정/원본 변경,
중복 요청, 데이터 보존, 실행·제출 실패를 우선 확인해라.
각 지적은 요구 ID·심각도·파일:줄·재현 입력/행동·기대/실제·최소 수정안을 담아라.
재현하지 못한 우려는 추정으로 표시해라.
취향 리팩터링과 추가 기능은 필수 결함과 구분해라.
검토자가 직접 제품 코드를 수정하지 말고 메인에게 결과를 반환해라.
```

수정 명령

```
검토 지적을 실제 요구·코드·재현 근거와 대조해라.
필수 요구 위반은 수정하고, 나머지는 사용자 영향과 남은 시간으로 우선순위를 정해라.
채택한 지적은 원 구현자에게 맡겨 최소 수정하고 같은 재현 절차로 재검증해라.
검토자가 관련 항목과 영향 범위를 다시 확인하게 해라.
새로운 기능 확대는 보류하고, 남는 위험은 이유와 함께 기록해라.
수정 후 변경된 코드에 과거 검사 결과가 여전히 유효한지 확인해라.
```

종료 기준은 필수 요구 충족, 채택한 결함의 재검증, 제출 루트의 실제 실행이다. 검사 통과를 위해 검사를 삭제하거나 기준을 약화하지 않는다. 미해결 필수 요구는 완료로 바꾸지 않고 명시한다.

내가 직접 발견한 오류 전달

```
입력/행동: [재현 절차]
기대 결과: [업무 규칙에 따른 값/동작]
실제 결과: [관찰한 값/오류]
이 차이의 원인을 찾아 최소 수정하고, 같은 사례와 관련 경계값으로 확인해라.
```

화면을 확인할 도구가 없으면 에이전트는 실행 가능한 API/로직 검사와 내가 직접 수행할 UI 확인 절차를 준다. 내가 확인하지 않은 화면을 정상이라고 기록하지 않는다.

5. 마무리 · 내가 설명할 수 있게 만든다

남은 시간의 마지막 15~20%에 이 단계로 전환한다. 필수 요구가 남으면 실제 남은 분량과 함께 알리고, 선택 기능을 줄여 필수 구현과 검증을 우선한다.

최종 검증과 설명 준비 명령

```
현재 남은 시간은 [분]이다. 남은 필수 요구와 큰 위험부터 점검해라.
선택 기능 확대를 멈추고 제출 루트에서 필수 흐름을 확인해라.
유효한 기존 검사는 재사용하고, 변경·실패·미검증 영역은 검사해라.
실제 실행/미리보기 주소에서 필요한 입력→최종 결과와 오류 경로를 확인해라.

기존 설명 기록 또는 PROJECT_DEFENSE.md를 현재 코드·실행 근거로 갱신해라.
1. 무엇을 왜 만들었는지와 필수 요구
2. 실제 실행·배포 위치, 명령, 데이터 저장 위치
3. 실제 사용하는 외부 패키지와 내부 모듈의 역할
4. 입력→검증→조회→계산→저장→표시의 함수 연결
5. 알고리즘 선택 이유, 대안, 최적성 보장 여부
6. 설정/DB 변경·중복 요청·중간 실패 시 현재 동작
7. 추가 가정, 미구현·미검증, 가장 중요한 개선안

각 항목은 쉬운 설명 뒤에 파일:함수와 실제 검사 근거를 붙여라.
구현한 것과 앞으로 할 것을 분리하고, 측정하지 않은 수치를 만들지 마라.
가능하면 독립 검토자에게 설명과 코드가 다른 주장만 찾게 해라.
내가 직접 확인할 최종 UI·저장·제출 항목을 짧게 전달해라.
```

답을 가린 사전 리허설 명령

```
현재 구현에 관한 질문 5개만 먼저 줘. 모범답안과 힌트는 보여주지 마라.
배포·모듈·데이터 흐름에서 2개, 선택·변경·실패·가정에서 3개를 골라라.
내 답을 받은 뒤 실제 코드와 다른 주장과 빠진 한계를 짚어라.
막힌 부분은 작은 입력의 값 변화로 설명하고, 다른 사례로 다시 물어라.
실제 답변을 받지 않았다면 이해가 확인됐다고 기록하지 마라.
```

내 답변의 골격

“현재는 [방식]으로 구현했습니다. [요구/환경] 때문에 선택했고 [파일·함수]에서 처리합니다. [실제 검사]로 확인했습니다. 다만 [한계]가 있어 [개선]이 필요합니다.”

배포 설정 파일의 존재와 실제 배포 성공을 구분한다. 제공 미리보기만 사용했다면 그렇게 말한다. 비동기 문법과 내구성 있는 작업 큐, 실행 가능해와 최적해, 문서 생성과 본인 이해도 각각 구분한다.

6. 막혔을 때 · 흐름을 복구하는 짧은 명령

분석·계획이 길어질 때

지금까지 확인한 사실과 구현을 막는 미확인만 요약해라.
첫 사용자 흐름을 실행하는 데 필요한 최소 변경을 골라 바로 진행해라.
불필요한 전수 조사·문서 확대를 줄이고 나머지 필수 요구는 계획에 유지해라.

한 가지 답을 못 받아 전체가 멈출 때

미확인 조건에 실제로 의존하는 행동만 특정해라.
그 조건을 추측해 확정하지 말고, 독립적으로 구현·검사할 수 있는 부분은 계속해라.
결정이 필요한 구체 사례와 대안의 영향을 짧게 알려줘.

에이전트가 충돌하거나 작업이 중복될 때

현재 에이전트·작업 경로·편집 파일·실행 상태를 확인해라.
겹치는 편집은 조정하고 파일별 담당자를 하나로 정해라.
기존 변경을 보존하면서 공통 계약을 다시 전달하고 통합 순서를 확정해라.
상태를 모르는 실행은 먼저 확인하고 같은 작업을 다시 시작하지 마라.

새 세션으로 이어갈 때

현재 PLAN과 구현 설명 기록, 실제 변경·검사·실행 상태를 대조해라.
작업자별 경로·담당 파일·막힘·완료 결과, 실행 서버와 다음 행동을 확인해라.
이전 인계와 실제 코드가 다르면 현재 근거로 정정해라.
이미 확정된 요구와 선택은 유지하고 남은 작업부터 이어가라.

시간이 급격히 줄었을 때

남은 시간은 [분]이다. 선택 기능과 취향 개선을 중단해라.
미완료 필수 요구, 실행/저장 실패, 결과 오류를 우선순위로 정리해라.
지금 완성 가능한 경로를 진행하고, 미완료는 숨기지 말고 명시해라.
최종 확인과 내가 설명을 복습할 시간을 남겨라.

참고와 적용 범위

이 운영안은 사용자의 기존 방식과 공개 자료를 토대로 작성한 제안이며, 공식 출제 문항이나 성과가 입증된 최적 절차는 아니다. 별도 키트를 설치하지 않고 시작하도록 구성했다. 공개 저장소는 준비 문서를 가져오는 용도다. 실제 과제 코드·고객 답변·작업 기록은 이 저장소에 올리지 않는다.

- COFA의 구현 대조 설명: https://www.getcofa.com/en/probe

- 구현 문답 참고: https://github.com/leey00nsu/cofa-hackathon-kit

- 독립 작업 분할 참고: https://github.com/obra/superpowers

- 공식 서브에이전트 설명: https://learn.chatgpt.com/docs/agent-configuration/subagents

작성 기준: 2026-11-05. 현재 공식 지침·제공 도구·남은 시간에 맞춰 적용한다.